25. Josephus Problem using LinkedList in Java — Detailed Notes

Problem Statement

Given:

- n → Number of people standing in a circle.
- k → Every kth person is eliminated.

Find:The position/ID of the last surviving person.

Example

Input

n = 7
k = 3
People:0 1 2 3 4 5 6

Elimination Process

Start counting from 0.
0 → 1st person
1 → 2nd person
2 → 3rd person → REMOVE

Remaining: 0 1 3 4 5 6
Continue counting from the next person (3).
3 → 1st person
4 → 2nd person
5 → 3rd person → REMOVE

Remaining: 0 1 3 4 6

Continue:
1 removed
6 removed
4 removed
0 removed

Elimination Order

Round	Removed
1	2
2	5
3	1
4	6
5	4
6	0

Survivor:3

Approach 1: Using LinkedList + Iterator

Idea

Use: LinkedList + Iterator
to simulate circular elimination.

Why Iterator?

Iterator provides:

Traversal

it.next();

Removal during traversal

it.remove();

Continue from next person automatically

After removal: Iterator points to the next surviving person.

Iterator Position

Initially:

↓
0 1 2 3 4 5 6

Iterator is positioned:
BEFORE first element.

Algorithm

Step 1

Create LinkedList.

```
for(int i=0;i<n;i++)
    list.add(i);
```

Step 2

Create iterator.

```
Iterator<Integer> it = list.iterator();
```

Step 3

Move k times.

```
int count = 0;
while(count < k){
    if(!it.hasNext())
        it = list.iterator();
    it.next();
    count++;
}
```

Step 4

Remove kth person.

```
it.remove();
```

Step 5
Repeat until only one person remains.
while(list.size() > 1)

Complete Code

```
static int josephus(int n,int k){

   LinkedList<Integer> list = new LinkedList<>();

   for(int i=0;i<n;i++)
      list.add(i);

   Iterator<Integer> it =list.iterator();

   while(list.size() > 1){
      int count = 0;
      while(count < k){

         if(!it.hasNext())
            it = list.iterator();

         it.next();
         count++;
      }
      it.remove();
   }
   return list.getFirst();
}
```

Why k Moves?
Iterator initially starts:
↓
0 1 2 3
For: k = 3

Need:
next() → 0
next() → 1
next() → 2

Hence: k next() calls.

Iterator remove()
Important: it.remove();
Removes: LAST element returned by next().
Example:
it.next(); // returns 2
it.remove();

Removes: 2

Circular Traversal

Java LinkedList is NOT circular.

When iterator reaches the end:

```
if(!it.hasNext())
    it = list.iterator();
```

Reset:
↓
0 1 3 4 6

This simulates: Circular traversal.

Dry Run

n = 7, k = 3
Initial: 0 1 2 3 4 5 6
Remove: 2
Remaining: 0 1 3 4 5 6
Remove: 5
Remaining: 0 1 3 4 6
Remove: 1
Remaining: 0 3 4 6
Remove: 6
Remaining: 0 3 4
Remove: 4
Remaining: 0 3
Remove: 0
Remaining: 3
Answer: 3

Time Complexity

Traversal: O(k)
Removal: O(1)
Total: O(n × k)

Approach 2: Without Iterator

Idea

Maintain current index.
Use modulo for circular traversal.

Code

```
static int josephus(int n,int k){
```

```
    LinkedList<Integer> list = new LinkedList<>();

    for(int i=0;i<n;i++)
        list.add(i);

    int index = 0;

    while(list.size() > 1){
        index = (index + k - 1)% list.size();
        list.remove(index);
    }

    return list.get(0);
}
```

Understanding the Formula

index = (index + k - 1) % list.size();

index

Represents: Person from whom counting starts.

k − 1: Represents: Number of jumps required.

Why k − 1?

Suppose:Current Person = 1st person

For:k = 3

Count:
Current → 1st
Next → 2nd
Next → 3rd

Need: 2 jumps.
Therefore: k − 1

Modulo (%)

Provides: Circular traversal.

Example:

Size = 5
Indices: 0 1 2 3 4
After: 4 → 5 → 6 → 7

Modulo converts:

5 % 5 = 0
6 % 5 = 1
7 % 5 = 2

Hence: End → Beginning

Formula Breakdown

Means: (index + k - 1)

Move k−1 jumps from current position.
Then: % list.size()
Means: Wrap around the circle.

Interview Thinking Process

Step 1:What does index represent? **Ans:** Current person.
Step 2:How many jumps are needed? **Ans:** k−1.
Step 3: Is the problem circular? **Ans:** Yes.
Step 4: How to implement circular movement? **Ans:** Modulo.
Step 5: Formula: (index + k − 1) % size

Comparison

Feature	Iterator Approach	Index Approach
Easy to understand	Moderate	Easy
Demonstrates Iterator	Yes	No
Circular traversal	Reset iterator	Modulo
Removal	Iterator.remove()	remove(index)
Complexity	O(n × k)	$O(n^2)$ for LinkedList

Important Interview Points

1. Josephus is a circular elimination problem.

2. LinkedList + Iterator is commonly used to demonstrate:
    - Iterator traversal
    - Iterator removal.

3. Iterator starts:BEFORE first element.
   Hence: Need k next() calls.

4. Index approach starts ON current person.
   Hence: Need k−1 jumps.

5. Modulo is the standard technique for: Circular traversal.

Quick Revision

Concept	Key Point
Problem	Eliminate every kth person
Data Structure	LinkedList
Iterator Use	Traversal + O(1) removal
Circular Traversal	Reset iterator / Modulo
Iterator Solution	k moves
Index Solution	k−1 jumps
Formula	`(index + k - 1) % size`
Survivor	Last remaining person
Complexity	O(n × k)

Great interview question. Josephus Problem can be solved using multiple data structures, and each demonstrates different concepts.

Data Structure / Approach	Time Complexity	Why Use It?
LinkedList + Iterator	**O(n × k)**	Demonstrates Iterator and safe removal
ArrayList + Index Formula	**O(n²)**	Easiest to understand
Circular Linked List	**O(n × k)**	Most natural representation of the problem
Queue (Deque)	**O(n × k)**	Simple rotation-based solution
Recursion (Mathematical)	**O(n)**	Most commonly asked interview solution
Dynamic Programming	**O(n)**	Bottom-up version of recursion
Segment Tree	**O(n log n)**	Efficient for large k and n
Fenwick Tree (BIT)	**O(n log n)**	Efficient order-statistics approach
Order Statistic Tree / Balanced BST	**O(n log n)**	Supports kth alive person queries

---

1. LinkedList + Iterator

Used in your notes.

Iterator<Integer> it = list.iterator();
Advantages

- `Iterator.remove()` → O(1)
- Demonstrates Iterator concepts.

Complexity: O(n × k)

---

2. ArrayList + Index Formula

index = (index + k - 1) % list.size();
list.remove(index);

Advantages

- Very easy to derive.
- Great for explaining interview logic.

Problem

list.remove(index);
causes shifting.

Complexity: O(n²)

---

3. Circular Linked List

Actually build:

1 → 2 → 3 → 4
↑ ↓
← ← ← ← ←

Algorithm

Move:
k−1 nodes
Delete kth node.
Continue.

Complexity: O(n × k)

---

4. Queue / Deque

Example: 1 2 3 4 5
For: k = 3

Rotate:

```
for(int i=1;i<k;i++)
   q.offer(q.poll());

q.poll();
```

Dry Run

1 2 3 4 5
Rotate: 2 3 4 5 1
Rotate: 3 4 5 1 2

Remove: 3
Complexity: O(n × k)

---

5. Recursion (Most Important)

Interview favorite.

Formula:
J(1,k) = 0
J(n,k) = (J(n−1,k) + k) % n

Example:

n = 7, k = 3

J(1)=0
J(2)=(0+3)%2=1
J(3)=(1+3)%3=1
J(4)=(1+3)%4=0
J(5)=(0+3)%5=3
J(6)=(3+3)%6=0
J(7)=(0+3)%7=3

Answer: 3
Complexity: O(n)

---

6. Dynamic Programming

Bottom-up version of recursion.

```
int ans = 0;
for(int i=1;i<=n;i++)
   ans = (ans + k) % i;
return ans;
```

Complexity: O(n)
Space: O(1)

---

7. Segment Tree

Useful when: n is huge

Need to find: kth alive person efficiently.
Supports: Find kth person → O(log n)
Delete person → O(log n)
Total: O(n log n)

---

8. Fenwick Tree (BIT)

Maintain:
Alive = 1
Dead = 0

Use prefix sums to find: kth alive person

Complexity: O(n log n)

Which one should you mention in interviews?

Beginner / DSA Round

Mention:
1. ArrayList + modulo logic
2. LinkedList + Iterator
3. Recursive formula

Strong Candidate

Mention: Recursive O(n)
Segment Tree O(n log n)
for optimization.

Which one should YOU remember?

Since you're preparing DSA interviews:

Level	Remember
Easy	ArrayList + `(index+k-1)%size`
Medium	LinkedList + Iterator
Important	Recursive Formula
Advanced	Segment Tree

Interview Answer

> "Josephus can be solved using LinkedList, ArrayList, Circular Linked List, Queue, or mathematically using recursion. The optimal interview solution is the recursive relation `J(n,k) = (J(n-1,k)+k)%n`, which runs in O(n). LinkedList + Iterator is

useful to demonstrate iterator traversal and removal concepts, while Segment Trees can optimize the problem further to O(n log n)."

This answer usually impresses interviewers because it shows **progression from simulation → mathematical optimization → advanced data structures**.